Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

«Изучение принципов поиска уязвимостей в программном обеспечении без исходных кодов»

по дисциплине «Модели безопасности компьютерных систем»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Жуковский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания **3**

2 Теоретические сведения **6**

3 Результаты работы **8**

3.1 Анализ исследуемого приложения vuln3.exe и структуры конфига 8

3.2 Статический анализ в IDA Pro и исследование уязвимости 8

3.3 Измерение покрытия кода с помощью DynamoRIO 8

3.4 Разработка фаззера, поиск уязвимости и анализ краша 8

3.5 Разработка IDAPython-скрипта для поиска опасных функций 8

4 Выводы **16**

ПРИЛОЖЕНИЕ А **17**

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Одним из наиболее распространенных способов поиска ошибок и уязвимостей в программном обеспечении в условиях отсутствия исходного кода является фаззинг. В данной лабораторной работе предлагается осуществить мутационный фаззинг формата файла. Так как формат конфигурационного файла не известен, но есть пример правильного файла, то следует осуществлять фаззинг путем модификации (мутации) оригинального файла. Разрабатываемый фаззер должен иметь режим работы, основанный на осуществлении мутаций входных данных с учетом покрытия кода программы.

В рамках лабораторной работы необходимо получить в соответствии со своим вариантом исполняемый файл, конфигурационный файл и динамические библиотеки. Лабораторная работа выполняется СТРОГО в соответствии со своим вариантом. В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1. Изучить основные типы ошибок в программном обеспечении (целочисленное переполнение, отсутствие проверки длины копируемых данных, переполнение буфера и другие).

2. Получить у преподавателя файлы в соответствии со своим вариантом.

3. Реализовать программу, осуществляющую фаззинг формата файла.

4. Реализованная программа должна осуществлять следующие действия:

— осуществлять изменение оригинального файла (однобайтовая замена, замена нескольких байт, дозапись в файл);

— заменять байты на граничные значения (0x00, 0xFF, 0xFFFF, 0xFFFFFF, 0xFFFFFFFF, 0xFFFF/2, 0xFFFF/2+1, 0xFFFF/2-1 и т.д.);

— иметь автоматический режим работы, при котором производится последовательная замена байт в файле;

— находить в файле символы, разделяющие поля (“,:=;”);

— расширять значения полей в файле (дописывать в конец, увеличивать длину строк в файле);

— осуществлять запуск исследуемой программы;

— используя средство динамической бинарной инструментации (DBI) (Intel Pin / DynamoRIO) осуществлять измерение покрытия кода во время фаззинга;

— реализовать режим работы фаззера с обратной связью на основе покрытия кода, основанный на сохранении измененных байт в файле с учетом их влияния на покрытие кода программы;

— обнаруживать возникновение ошибки в исследуемом приложении;

— получать код ошибки и состояние стека и регистров на момент возникновения ошибки;

— логировать в файл информацию о произошедших ошибках и соответствующих им входных параметрах (произведенные замены). 5. Разработать IDC/IDAPython-скрипт, осуществляющий следующие действия:

— поиск в программе функций ввода данных (fread, fscanf, read, fgets, …);

— поиск вызовов небезопасных функций (strcpy, sprintf, strncpy, memcpy, memmove, …).

6. Разработанный IDC/IDAPython-скрипт должен выводить следующую информацию:

— название найденной функции;

— адрес, откуда вызывается данная функция.

7. Используя разработанную программу для фаззинга формата файлов осуществить поиск в выданной программе уязвимости (целочисленное переполнение, отсутствие проверки длины данных), приводящей к переполнению буфера.

8. Сформировать файл, приводящий к краху исследуемого приложения.

9. Изучить с использованием дизассемблера и отладчика структуру программы и найти участок кода с уязвимостью. Провести исследование найденной уязвимости и условий ее эксплуатации.

10. В отчете необходимо привести следующую информацию:

— исходный код программы, осуществляющей фаззинг формата файла;

— описание методов, применяемых при фаззинге;

— участок кода с найденной уязвимостью (из дизассемблера), с комментариями;

— восстановленный исходный код (на языке C) участка с уязвимостью;

— описание найденной уязвимости (тип ошибки, причины, ограничение при эксплуатации);

— размер буфера, адрес начала буфера, адрес сохраненного на стеке адреса возврата (return address, за буфером);

— формат файла, используемого для эксплуатации, с указанием значения полей, приводящих к краху программы;

— описание уязвимости и механизмов ее эксплуатации;

— исходный код разработанных IDC/IDAPython-скриптов;

— описание возможного способа исправления уязвимости.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Поиск уязвимостей в программном обеспечении без исходных кодов выполняется методами статического и динамического анализа бинарных файлов. Одним из основных динамических методов является фаззинг (fuzzing, fuzz testing) — автоматизированное тестирование, при котором программе на вход подаётся большое количество специально сгенерированных или искажённых данных с целью вызвать аварийное завершение, зависание или иное аномальное поведение.

Основные виды фаззинга:

— Мутационный: новые входные данные создаются путём модификации уже существующих корректных примеров (замена байт, дозапись, изменение значений полей и т.д.). Именно этот вид используется в данной работе.

— Генерационный: входные данные создаются «с нуля» на основе описания формата или грамматики.

— Black-box: программа рассматривается как «чёрный ящик», отслеживается только факт краха.

— Grey-box (coverage-guided): дополнительно собирается информация о покрытии кода. Мутации, которые приводят к выполнению ранее недостижимых участков, сохраняются и используются для дальнейшего развития. Такой подход значительно эффективнее чисто случайного.

В работе применяется мутационный coverage-guided фаззинг формата файла. Корректный конфигурационный файл модифицируется (однобайтовые и многобайтовые замены, дозапись, подстановка граничных значений 0x00/0xFF/0xFFFF и т.д., расширение полей). Покрытие кода измеряется средствами динамической бинарной инструментации (DBI) — Intel Pin или DynamoRIO. Эти фреймворки вставляют аналитические процедуры в выполняющийся бинарный код и фиксируют достигнутые базовые блоки.

Наиболее характерные ошибки, выявляемые таким образом:

— отсутствие проверки длины копируемых данных и использование небезопасных функций (strcpy, sprintf, memcpy и др.);

— целочисленное переполнение при расчёте размеров буферов, приводящее к выделению меньшего объёма памяти, чем требуется;

— переполнение буфера (особенно стекового). При стековом переполнении данные затирают соседние области, включая сохранённый адрес возврата. Это приводит к краху программы и, при определённых условиях, позволяет изменить поток управления.

Для локализации потенциально опасных мест дополнительно применяется статический анализ (дизассемблер IDA Pro и скрипты IDC/IDAPython), в ходе которого ищутся вызовы функций ввода данных и небезопасных функций работы с памятью. Сочетание мутационного coverage-guided фаззинга, DBI и последующего анализа крашей и дизассемблированного кода позволяет обнаруживать уязвимости типа целочисленного переполнения и недостаточной проверки длины, приводящие к переполнению буфера.

3 РЕЗУЛЬТАТЫ РАБОТЫ

Для выполнения работы был изучен файл vuln3.exe. Для его запуска нужны библиотека func.dll и конфигурационный файл config_3. Программа выводит версию 0.3, номер варианта 3, сообщение о вызове функции из библиотеки и два числа из заголовка: shell_len 604 и dst_len 2500. Затем печатается строка buffer, прочитанная из конфига.

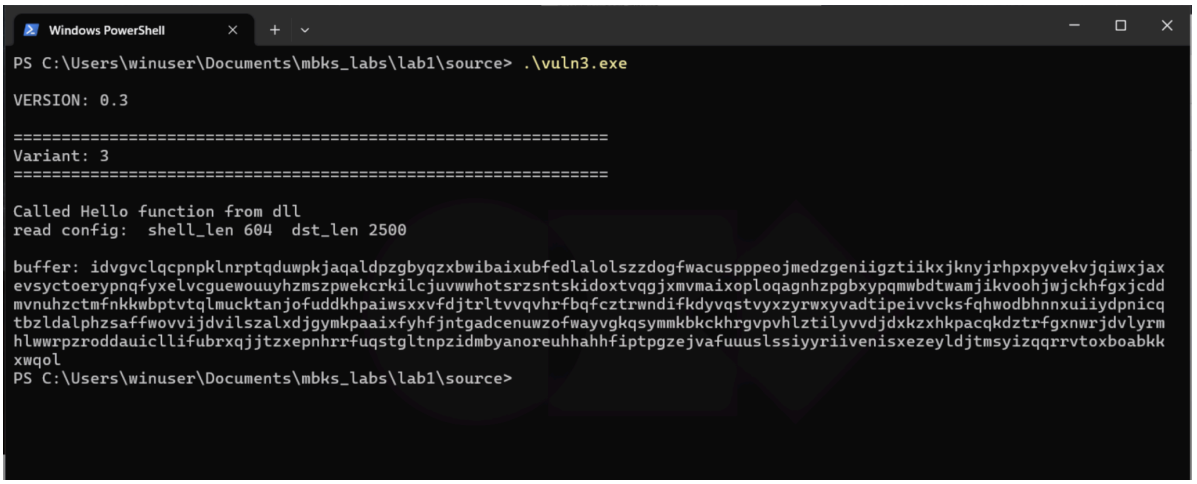


Рисунок 1 – Выполнение файла vuln3.exe

Файл config_3 имеет размер 652 байта. Первые 48 байт являются заголовком. По смещению 0 лежит байт варианта, его значение 0x03. Если этот байт изменить, программа завершится с сообщением variant does not match и до разбора строки не дойдёт. По смещению 4 записано поле shell_len, байты 5C 02 00 00, то есть 604. Это размер файла без заголовка: 652 − 48 = 604. По смещению 8 записано поле dst_len, байты C4 09 00 00, то есть 2500 (0x9C4). Это число совпадает с размером буфера, который программа заполняет через memset, но при копировании строки оно не проверяется. По смещению 48 (0x30) находится маркер /start. Сразу после него идёт текст длиной 598 байт. Именно этот текст передаётся в небезопасное копирование.

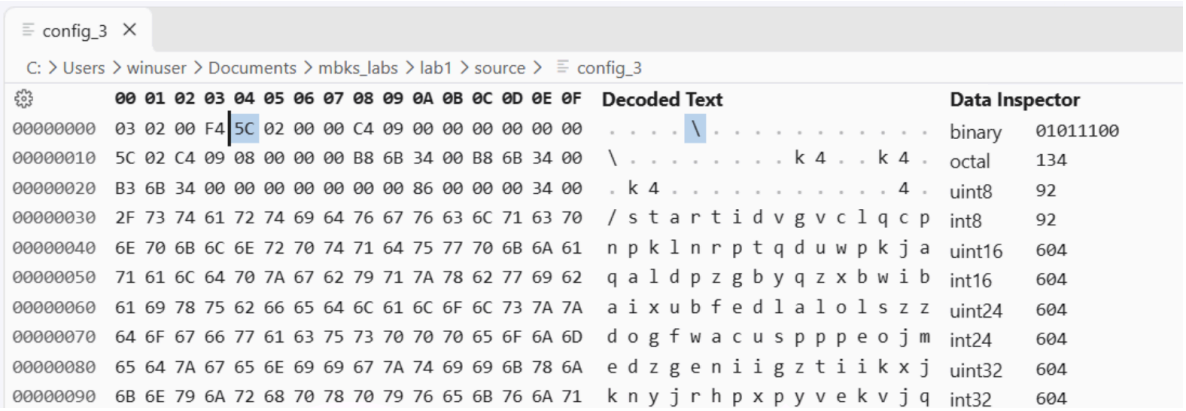


Рисунок 2 – shell_len в config_3

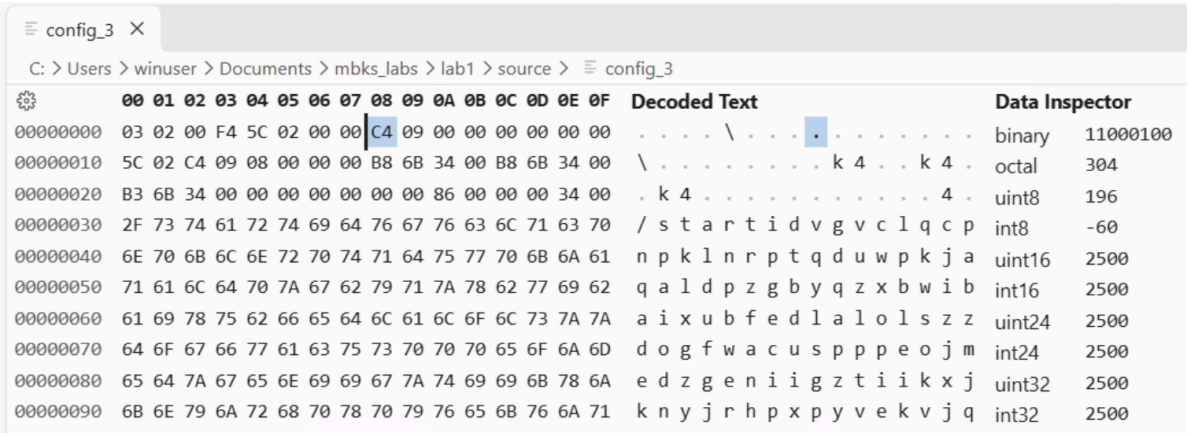


Рисунок 3 – dst_len в config_3

При разборе vuln3.exe в IDA Pro найдена функция vuln_func по адресу 0x0040158D. Она выделяет на стеке кадр размером 0x9E8 байт и вызывает memset с длиной 0x9C4 (2500 байт) и символом b. Начало буфера находится по адресу ebp-0x9D0. Сразу после memset тот же буфер передаётся в strcpy. Длина источника не сравнивается ни с 2500, ни с полем dst_len из файла. strcpy копирует байты до нулевого символа в сторону увеличения адресов, то есть к сохранённым ebp и адресу возврата.

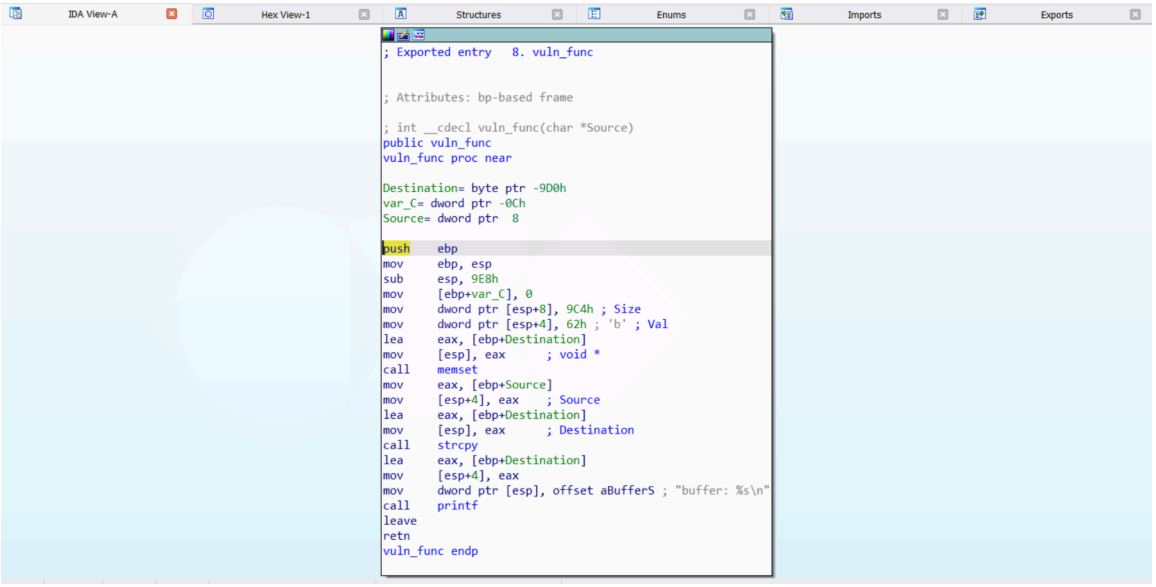


Рисунок 4 – Уязвимость в программе

Сохранённый ebp лежит на смещении 0x9D0 (2512 байт) от начала буфера. Адрес возврата лежит на смещении 0x9D4 (2516 байт), в ячейке [ebp+4]. Между концом области memset и сохранённым ebp остаётся 12 байт, поэтому заполнение буфера символом b само по себе кадр не затирает. Переполнение возникает в strcpy.

```
1  int __cdecl vuln_func(char *Source)
2  {
3    char Destination[2500]; // [esp+18h] [ebp-9D0h] BYREF
4    int v3; // [esp+9DCh] [ebp-Ch]
5
6    v3 = 0;
7    memset(Destination, 98, sizeof(Destination));
8    strcpy(Destination, Source);
9    return printf("buffer: %s\n", Destination);
10 }
```

Рисунок 5 – Псевдокод функции vuln_func

Строка попадает в эту функцию следующим путём. read_config по адресу 0x004015E8 открывает config_3, узнаёт размер через fseek и ftell и читает файл целиком вызовом fread с адреса 0x0040169A. В конец буфера дописывается нулевой байт. Если размер больше 0x2F, первые 0x30 байт копируются в заголовок через memcpy, вызов с адреса 0x004016C7. Указатель на байт со смещением 0x30

передаётся в parse_string по адресу 0x004013E0. Эта функция ищет подстроку /start. Если маркер найден, она вызывает саму себя для хвоста, который начинается сразу после /start. Если маркера в оставшейся строке уже нет, вызывается vuln_func.

Для измерения покрытия использован DynamoRIO и клиент drcov. Запуск выполняется командой drrun.exe -t drcov -dump_text -logdir cov -- vuln3.exe из каталога с vuln3.exe, func.dll и config_3. В журнале есть таблица модулей. Каждому модулю присваивается id. Модуль 0 — vuln3.exe, модуль 1 — func.dll. Ниже, в BB Table, для каждого выполненного базового блока записаны id модуля, смещение от базы модуля и размер блока в байтах.

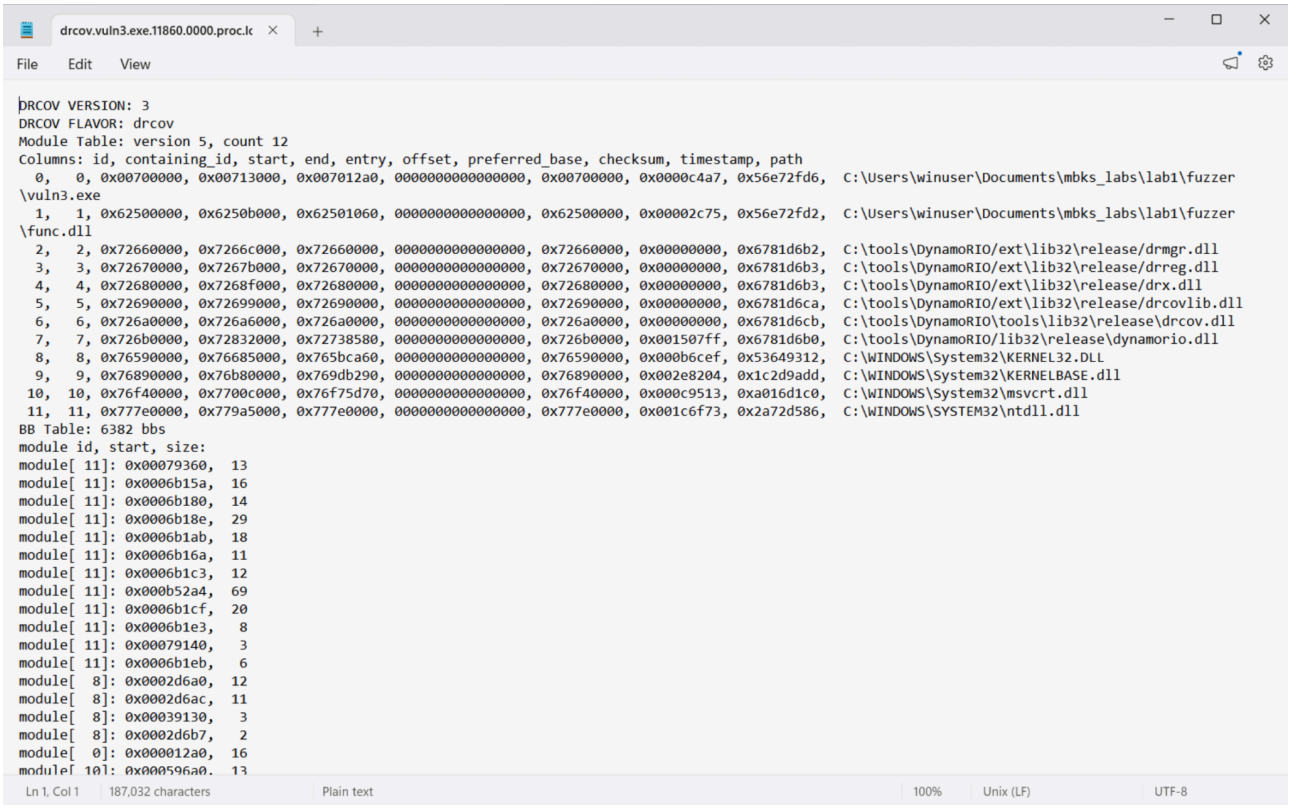


Рисунок 6 – Результат работы утилиты DynamoRIO

Смещение из журнала переводится в адрес IDA прибавлением базы образа 0x400000. Например, блок module[0] со смещением 0x12A0 и размером 16 байт открывается в IDA клавишей G по адресу 0x004012A0. Так проверяется, что записанный в логе блок относится к коду программы. Фаззер считает такие блоки и для vuln3.exe, и для func.dll. Если изменённый конфиг дал новые блоки, он сохраняется в каталог corpus.

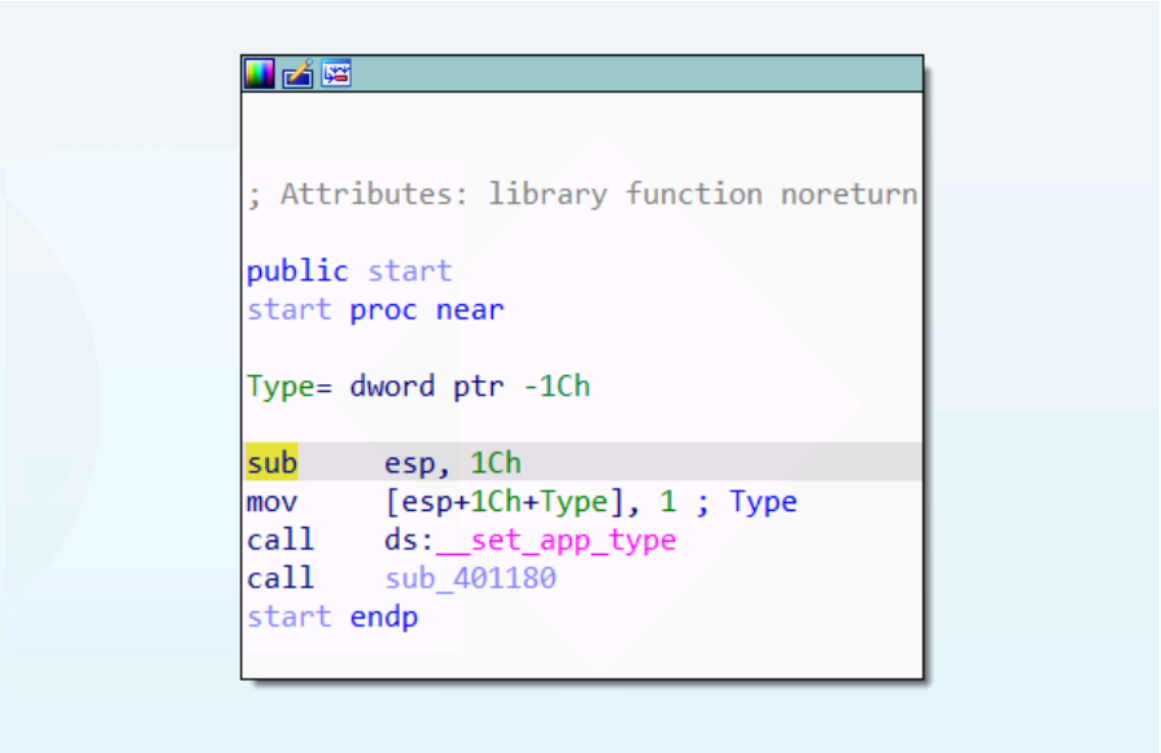


Рисунок 7 – Поиск выполненного блока программы vuln3.exe

Фаззер написан на Go и управляется меню. Он меняет рабочую копию config_3 и не трогает файл в каталоге source, пока не выбран пункт 10. Пункт 2 заменяет один байт, пункт 3 заменяет несколько байт, пункт 4 дописывает байты в конец. Граничные значения для ширины 1, 2, 3 и 4 байта — это 0, максимум (0xFF, 0xFFFF, 0xFFFFFF, 0xFFFFFFFF), половина максимума, половина плюс один и половина минус один. В файл они записываются в порядке little-endian. Пункт 5 подставляет эти значения по последовательным смещениям. Пункт 7 ищет разделители «,», «:», «=» и «;». В выданном config_3 их нет, поэтому пункт 8 вставляет байты сразу после /start. Пункт 6 запускает цель под отладчиком. При исключении в каталог crashes записываются изменённый файл, код ошибки, регистры и 256 байт стека. Пункт 9 повторяет мутации и оставляет в corpus только те входы, которые увеличили покрытие.

Для поиска ошибки выбран пункт 8 с длиной 2000. Фаззер вставил 2000 байт 0x41 по смещению 54, то есть сразу после /start, и запустил программу. Отладчик зафиксировал исключение C0000005. Повторного запуска под DynamoRIO после падения нет.

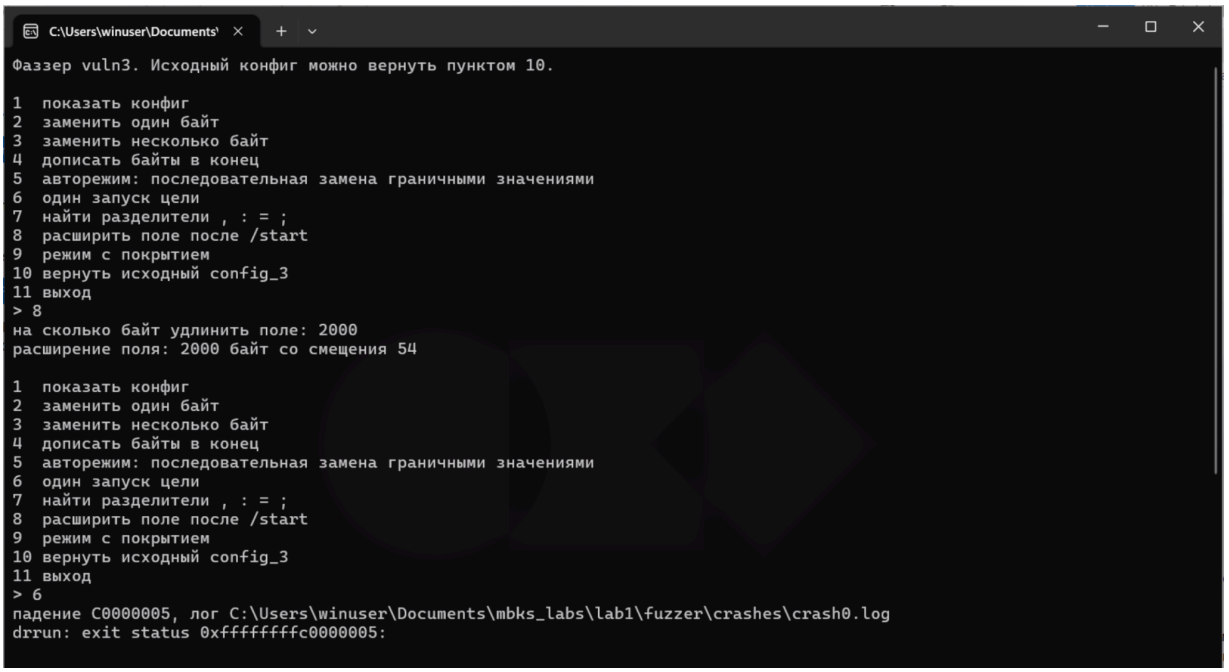


Рисунок 8 – Запуск фаззера и код исключения

В исходном файле после /start лежит 598 байт. На индексе 512 этой строки стоят восемь символов gltnpzid. Вставка 2000 байт A сдвигает их на индекс 2512 от начала буфера. Это место сохранённого ebp. Следующие четыре байта pzid попадают на адрес возврата. В снимке регистров это видно как EBP = 0x6E746C67 и EIP = 0x64697A70. Процесс обращается по адресу, который составлен из символов конфига, и получает отказ доступа.

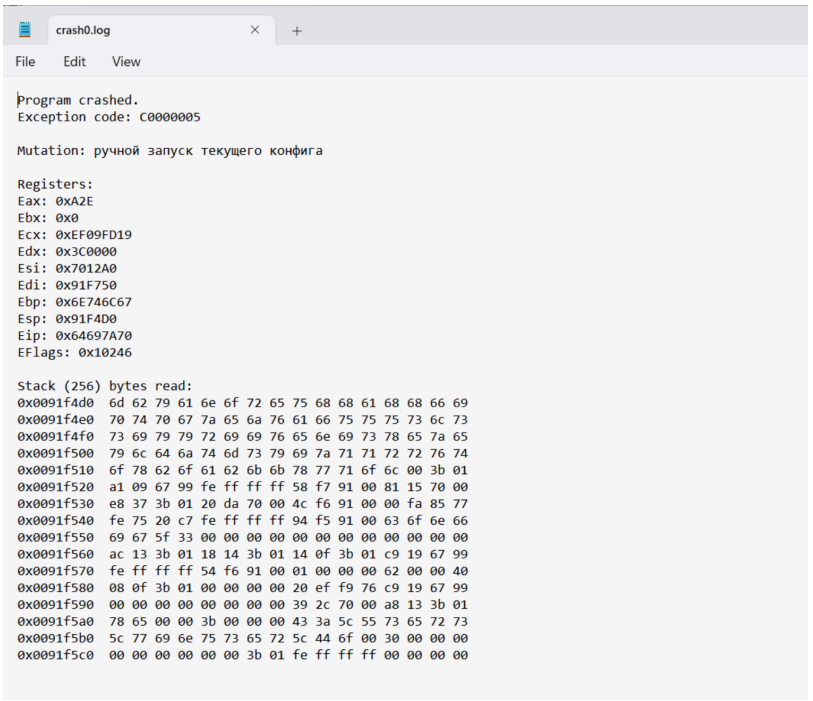


Рисунок 9 – Лог падения: код исключения, регистры и стек

Файл, приводящий к краху, устроен так. Байт 0 остаётся равным 0x03. Поля shell_len и dst_len можно не менять: программа их печатает, но длину strcpy по ним не ограничивает. Маркер /start на смещении 48 сохраняется. Сразу после него записываются 2000 байт 0x41. Размер файла становится 2652 байта. Более короткая вставка тоже затирает адрес возврата, если длина строки после /start достигает 2520 байт. При исходных 598 байтах для этого достаточно 1922 добавленных байт. В работе использовано проверенное значение 2000.

Для поиска опасных вызовов написан скрипт на IDAPython. Он ищет функции ввода fread, fscanf, read, fgets и небезопасные функции strcpy, sprintf, strncpy, memcpy, memmove. Для каждой найденной функции печатаются имя и адрес вызова. В vuln3.exe скрипт показывает fread по адресу 0x0040169A, sprintf по адресу 0x0040154E, strcpy по адресу 0x004015CB и memcpy по адресу 0x004016C7. Исходный текст скрипта приведён в приложении.

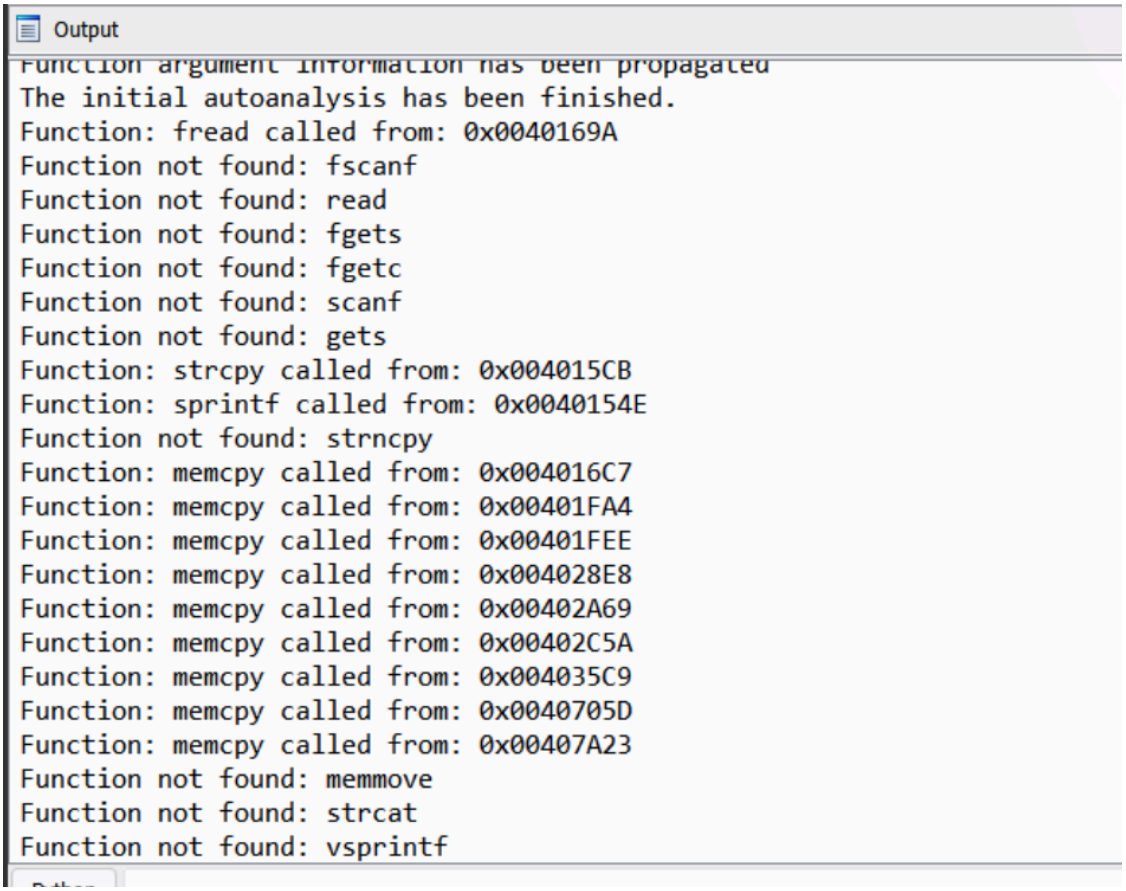


Рисунок 10 – Поиск функций ввода и небезопасных функций

Пункт 6 меню на исходном конфиге измеряет покрытие через DynamoRIO и выводит число базовых блоков vuln3.exe и func.dll. На неизменённом файле это покрытие принимается за базу. В режиме пункта 9 каждая мутация сравнивается с базой. Файл сохраняется только тогда, когда появились новые блоки.

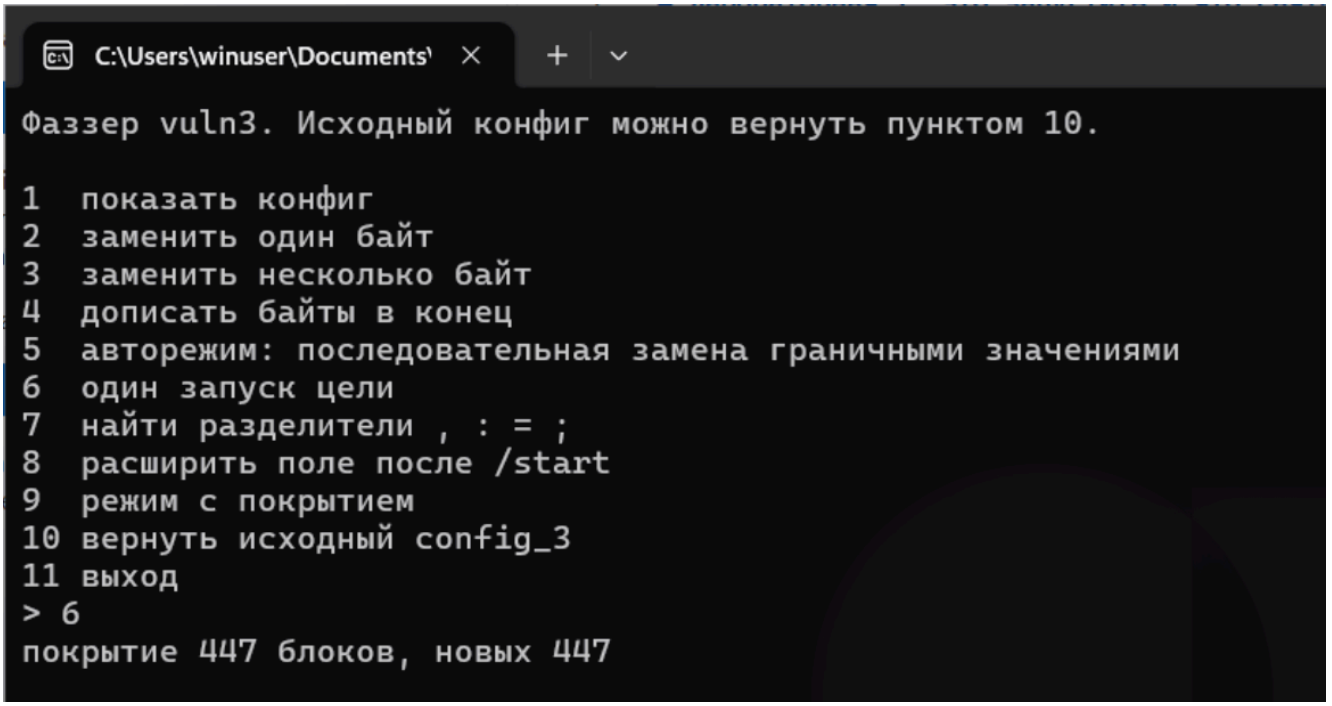


Рисунок 11 – Измерение покрытия из фаззера

4 ВЫВОДЫ

В работе выполнен поиск уязвимости в vuln3.exe без исходных текстов. По корректному config_3 написан мутационный фаззер на Go: однобайтовые и многобайтовые замены, допись, граничные значения шириной 1–4 байта в порядке little-endian, поиск разделителей и расширение поля после /start. Запуск идёт под отладчиком, при исключении сохраняются код, регистры, 256 байт стека и сам файл. Покрытие базовых блоков vuln3.exe и func.dll снимается клиентом DynamoRIO drcov. Отдельно скрипт IDAPython выписывает места вызова функций ввода и небезопасных функций работы с памятью.

Найденная ошибка — копирование без проверки длины. Функция vuln_func заполняет memset буфер в 2500 байт и затем переносит в него строку через strcpy. Поля shell_len и dst_len на это копирование не влияют. Строка длиннее 2512 байт затирает сохранённый ebp, строка длиной 2520 байт затирает и адрес возврата. Защиты стека нет, поэтому при возврате из функции в EIP попадают байты из файла. Для выданного конфига это получено вставкой 2000 байт 0x41 сразу после /start: регистры стали EBP = 0x6E746C67 и EIP = 0x64697A70, код исключения C0000005.

У режима с покрытием на этой ошибке мало помощи. Ветка strcpy выполняется уже на исходном файле, удлинение строки новых базовых блоков не добавляет, а падение в корпус не кладётся. Одна случайная мутация пункта 9 удлиняет поле не больше чем на 24 байта, поэтому до порога около 1922 байт она сама не добирается. Краш получен направленной мутацией пункта 8. Допись в конец файла для этого дефекта тоже бесполезна: исходный хвост уже заканчивается нулём, и strcpy до дописанных байт не доходит. Фаззер должен быть собран как 32-битный (GOARCH=386), иначе снимок регистров цели читается неверно.

Исправление состоит в замене strcpy на копирование с явной верхней границей размера стекового буфера. Сверять длину с dst_len из файла недостаточно: это поле контролирует тот, кто готовит конфиг.